services\analysis\cwe\utils.py

```
"""
CWE Utility Functions
Helper functions for CWE processing and analysis
"""
# Type hints for function parameters and return values
from typing import Dict, List, Tuple, Optional
# Centralized CWE constants and category mappings
from .constants import CWE_TITLES, EXTENDED_CWE_TITLES, CWE_CATEGORIES

def get_cwe_title(cwe_code: str) -> str:
"""Get human-readable title for CWE code"""
# Cascading lookup: primary titles, then extended, then fallback to code
return (CWE_TITLES.get(cwe_code) or
EXTENDED_CWE_TITLES.get(cwe_code) or
cwe_code)

def normalize_cwe_code(cwe_input: str) -> str:
"""Normalize CWE input to standard format (CWE-123)"""
# Handle empty or None input
if not cwe_input:
return ""

# Clean input: remove whitespace and standardize case
cwe_clean = cwe_input.strip().upper()

# Handle different input formats and convert to standard CWE-123
if cwe_clean.startswith('CWE-'):
# Already in standard format
return cwe_clean
elif cwe_clean.startswith('CWE'):
# Missing hyphen: CWE123 -> CWE-123
number_part = cwe_clean[3:]
if number_part.isdigit():
return f"CWE-{number_part}"
elif cwe_clean.isdigit():
# Just number: 123 -> CWE-123
return f"CWE-{cwe_clean}"

# Return original if normalization not possible
return cwe_input

def categorize_cwe(cwe_code: str) -> Optional[str]:
"""Categorize a CWE code into a broader category"""
# Search through all defined categories for CWE membership
for category, cwes in CWE_CATEGORIES.items():
if cwe_code in cwes:
return category
# Return None if CWE doesn't match any category
return None

def get_cwe_category_summary(cves: List[Dict]) -> Dict[str, int]:
"""Get summary of CVEs by CWE category"""
category_counts = {}

# Process each CVE for category classification
for cve in cves:
cwe = cve.get('CWE')
if cwe:
category = categorize_cwe(cwe)
if category:
# Count categorized CWEs
category_counts[category] = category_counts.get(category, 0) + 1
else:
# Count uncategorized CWEs as 'other'
category_counts['other'] = category_counts.get('other', 0) + 1

return category_counts

def extract_cwe_number(cwe_code: str) -> Optional[int]:
"""Extract numeric part from CWE code"""
try:
# Handle different CWE formats to extract numeric ID
```

```
if cwe_code.startswith('CWE-'):
return int(cwe_code[4:])  # CWE-123 -> 123
elif cwe_code.startswith('CWE'):
return int(cwe_code[3:])  # CWE123 -> 123
elif cwe_code.isdigit():
return int(cwe_code)      # 123 -> 123
except (ValueError, AttributeError):
# Return None for invalid input or conversion errors
pass
return None

def is_valid_cwe_code(cwe_code: str) -> bool:
"""Check if a string is a valid CWE code format"""
# Reject empty or None input
if not cwe_code:
return False

# Use normalization to standardize format, then validate
normalized = normalize_cwe_code(cwe_code)
return (normalized.startswith('CWE-') and
normalized[4:].isdigit() and
len(normalized) > 4)

def get_severity_color(severity: str) -> str:
"""Get color code for severity level"""
# Standard color mapping for security severity levels
severity_colors = {
'CRITICAL': '#f55855',  # Red for highest severity
'HIGH': '#f8a541',      # Orange for high severity
'MEDIUM': '#3b8ded',    # Blue for medium severity
'LOW': '#42d392',       # Green for low severity
'UNKNOWN': '#6b7280'    # Gray for unknown/unclassified
}
# Case-insensitive lookup with gray fallback
return severity_colors.get(severity.upper(), '#6b7280')

def format_cwe_for_display(cwe_code: str, include_title: bool = True) -> str:
"""Format CWE code for display purposes"""
# Handle missing CWE data
if not cwe_code:
return "No CWE"

# Standardize CWE format
normalized = normalize_cwe_code(cwe_code)

# Optionally include human-readable title
if include_title:
title = get_cwe_title(normalized)
# Only add title if it's different from the code
if title != normalized:
return f"{normalized}: {title}"

return normalized

def search_cwes_by_keyword(keyword: str) -> List[Tuple[str, str]]:
"""Search CWEs by keyword in titles"""
keyword_lower = keyword.lower()
matches = []

# Search through extended CWE titles for keyword matches
for cwe_code, title in EXTENDED_CWE_TITLES.items():
# Case-insensitive search in both code and title
if (keyword_lower in cwe_code.lower() or
keyword_lower in title.lower()):
matches.append((cwe_code, title))

# Sort results by CWE code for consistent ordering
return sorted(matches, key=lambda x: x[0])

def get_cwe_statistics(cves: List[Dict]) -> Dict[str, any]:
"""Get comprehensive CWE statistics from CVE list"""
# Initialize comprehensive statistics structure
stats = {
'total_cves': len(cves),
'cves_with_cwe': 0,
'unique_cwes': set(),          # Use set for automatic deduplication
'top_cwes': {},
```

```
'category_breakdown': {},
'severity_by_cwe': {}
}

cwe_counts = {}

# Process each CVE for comprehensive CWE analysis
for cve in cves:
cwe = cve.get('CWE')
severity = cve.get('Severity', 'UNKNOWN').upper()

# Only process CVEs with valid CWE codes
if cwe and is_valid_cwe_code(cwe):
stats['cves_with_cwe'] += 1
stats['unique_cwes'].add(cwe)  # Set automatically handles duplicates

# Count CWE occurrences for frequency analysis
cwe_counts[cwe] = cwe_counts.get(cwe, 0) + 1

# Track severity distribution per CWE for detailed analysis
if cwe not in stats['severity_by_cwe']:
stats['severity_by_cwe'][cwe] = {}
stats['severity_by_cwe'][cwe][severity] = stats['severity_by_cwe'][cwe].get(severity, 0)
+ 1

# Generate top 10 most frequent CWEs
stats['top_cwes'] = dict(sorted(cwe_counts.items(), key=lambda x: x[1],
reverse=True)[:10])

# Generate category breakdown using utility function
stats['category_breakdown'] = get_cwe_category_summary(cves)

# Convert unique CWEs set to count for JSON serialization
stats['unique_cwes'] = len(stats['unique_cwes'])

return stats
```